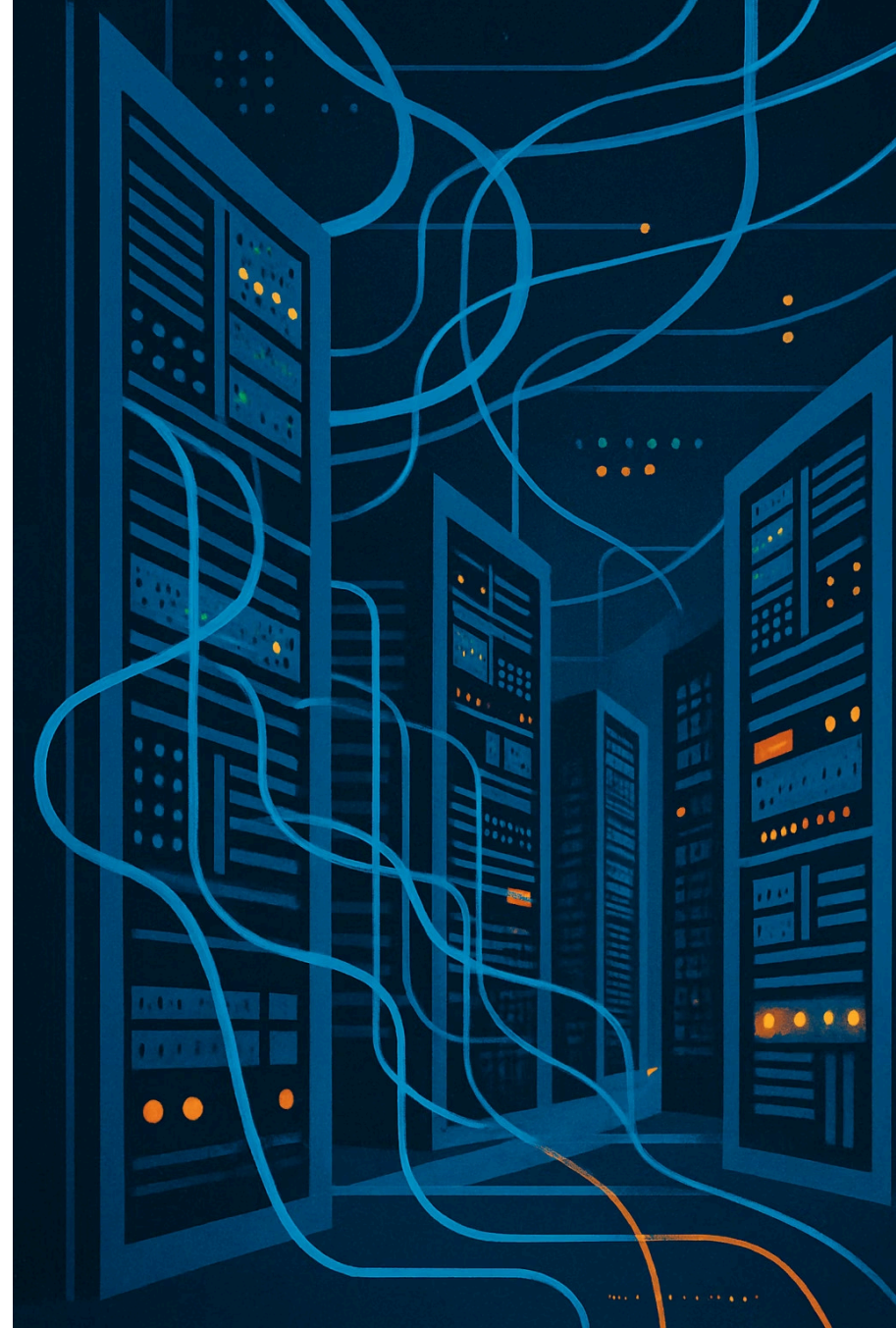


Securing AI Workloads in Kubernetes

LESSONS FROM SCALING STARTUPS

How to build systems where **secure is the default** and fast is still possible.



Who This Talk Is For



Startups Moving Fast

Teams shipping daily, scaling Kubernetes, and chasing product-market fit with security as an afterthought.



Teams Adding AI

Engineers deploying LLMs, inference endpoints, and model APIs without a clear threat model.



Security-Minded Builders

Leaders who want security to move at the speed of their platform, not slow it down.



THE SCENARIO

You've got a startup. Kubernetes is running. You finally get product-market fit ...

and then someone says: "Let's add AI."

Suddenly your most valuable asset is exposed through an API with almost no guardrails.

MEETING THE MENTOR

Security Wins When It Becomes Infrastructure, Not Process

Guardrails, not gates. Default secure platforms. Developer experience matters.

The Startup Default

Startups optimize for speed, not safety. And it works, until **scale**.

Identity Sprawl

OIDC chaos, over-permissioned service accounts.

Flat Networks

Everything talks to everything. No segmentation.

Secrets Everywhere

API keys in env vars, tokens in Git history.

Implicit Trust

"Inside the cluster" = trusted. No questions asked.

Startup Reality

Security Debt is Invisible Until it Isn't

✕ "We'll add mTLS later"

✕ "NetworkPolicies = done"

✕ "AI is just another microservice"

CROSSING THE THRESHOLD

AI Didn't Create New Problems

It amplified the ones we ignored.

Models

Intellectual property exposed
through inference APIs

Prompts

Sensitive data flowing
through unprotected channels

Endpoints

Expensive compute targets
ripe for abuse

The AI Threat Model

Prompt Injection

Adversarial inputs that manipulate model behavior, exfiltrate data, or bypass safety filters.

Token Abuse

Unauthorized callers burning through expensive compute. One compromised pod = \$10k overnight.

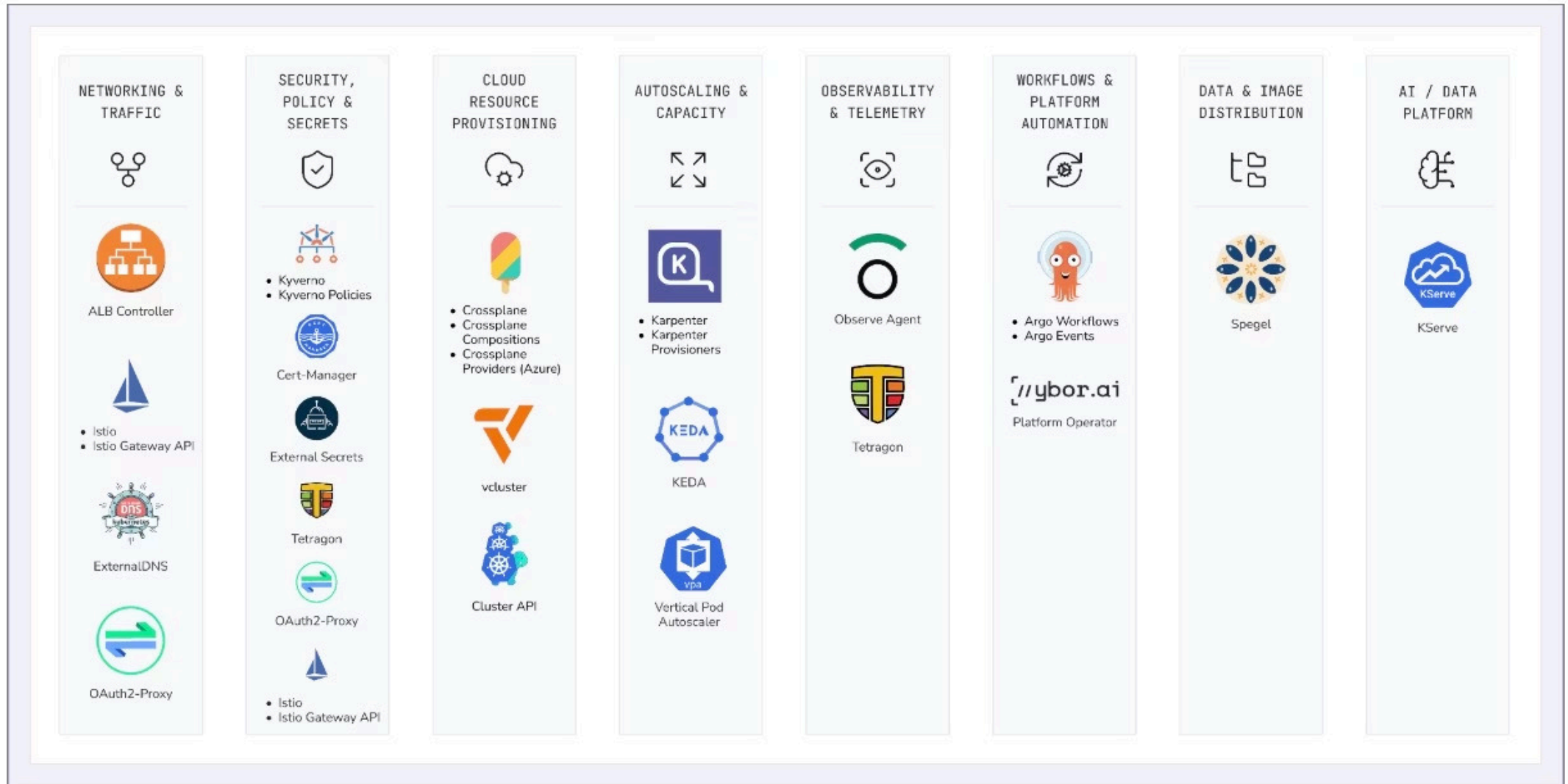
Model Scraping

Systematic extraction of model weights or behavior through repeated inference queries.

Lateral Movement

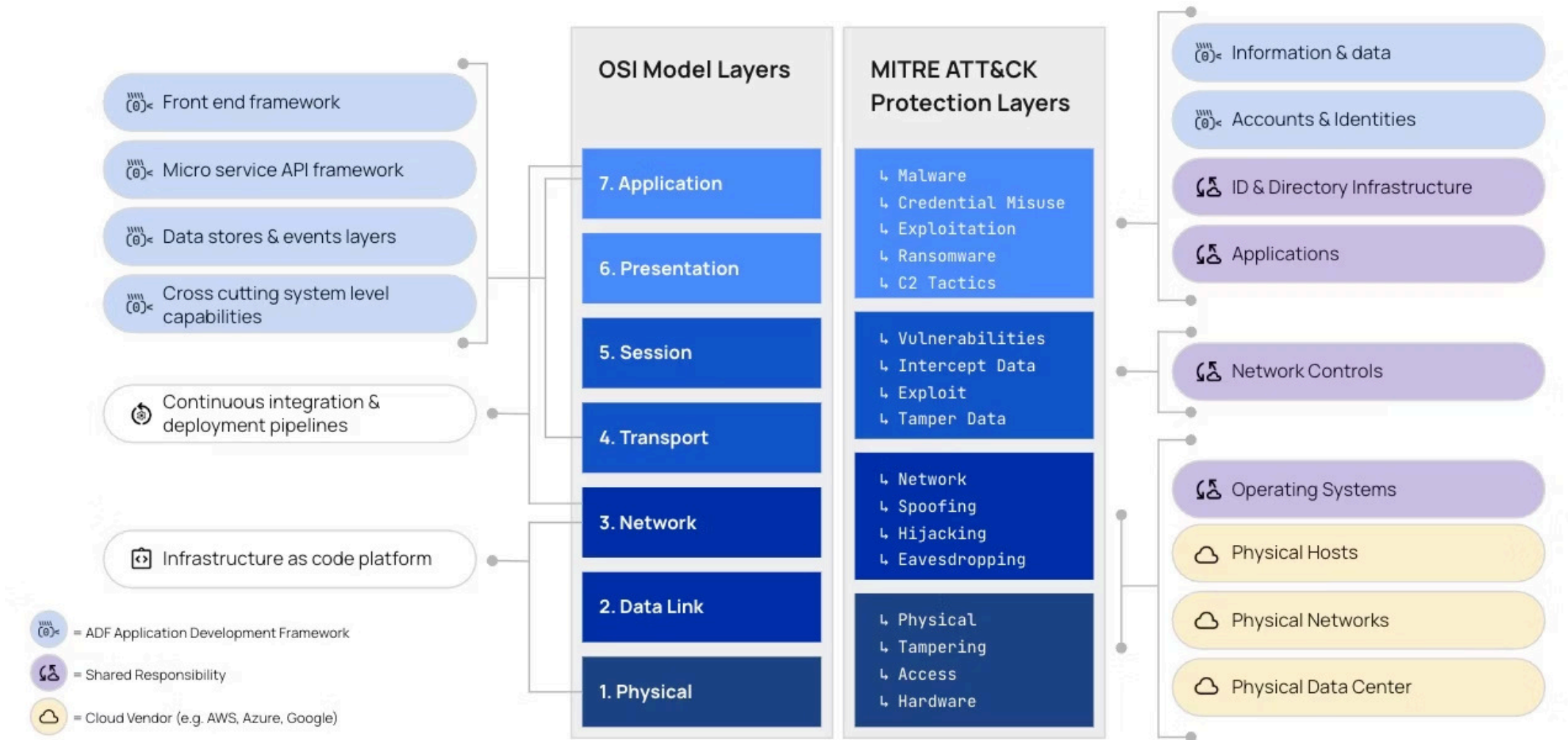
Compromised workloads pivoting to AI services. Flat networks make this trivial.

Cloud Native Computing Foundation (CNCF) Tools



The CNCF Landscape: <https://landscape.cncf.io>

Ybor.ai's shared security model



Security Wins When It Becomes Infrastructure

Identity-First Security

SPIFFE/mTLS workload identity over network trust

East-West Controls

Authorization policies + network policies with egress

AI-Specific Isolation

Model boundaries, not all services should see the model

Observability That Matters

Latency anomalies, token spikes, access patterns

Technical Deep Dive

Identity-First Security

No Identity = No Access



mTLS in Practice

mTLS ensures both sides prove who they are

Mutual Authentication

Both client and server present certificates. Neither side trusts the other without cryptographic proof.



Auto-Rotation

Certs rotate automatically every few hours in production. Compromise window shrinks to near zero.



Transparent to Apps

Service mesh handles the TLS layer. Your application code doesn't change. Devs stay happy.



eBPF-native performance

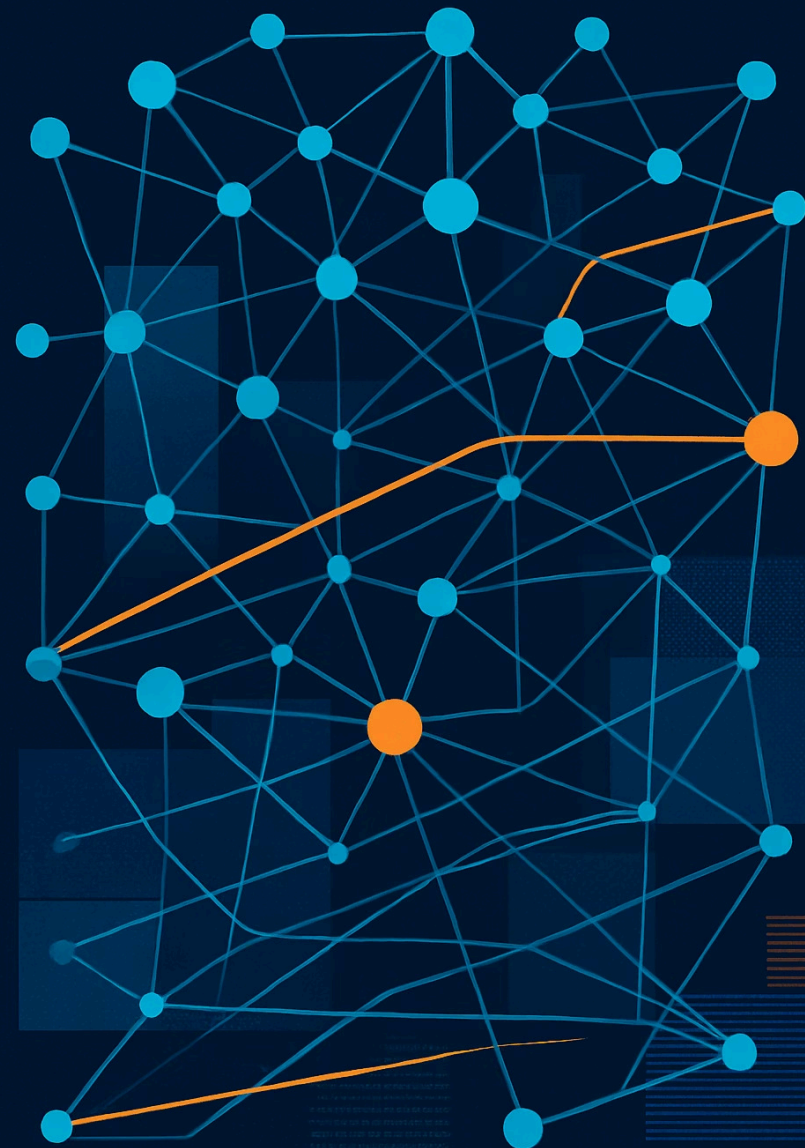


Broader Istio ecosystem

Technical Deep Dive

Controlling East-West Traffic

If a service doesn't need access, it doesn't get it.



East-West: The Overlooked Perimeter

✗ Flat Network, ✓ Explicit Trust Graph. **Default deny**, *then explicitly allow*

NetworkPolicy (L3/L4)

Controls **which pods** can communicate based on labels, namespaces, and ports. Enforced by CNI.

- Pod selector matching
- Namespace isolation
- Port-level rules

AuthorizationPolicy (L7)

Controls **what actions** are allowed: specific HTTP methods, paths, and headers. **Requires service mesh.**

- Method/path restrictions
- JWT principal binding
- Source workload identity

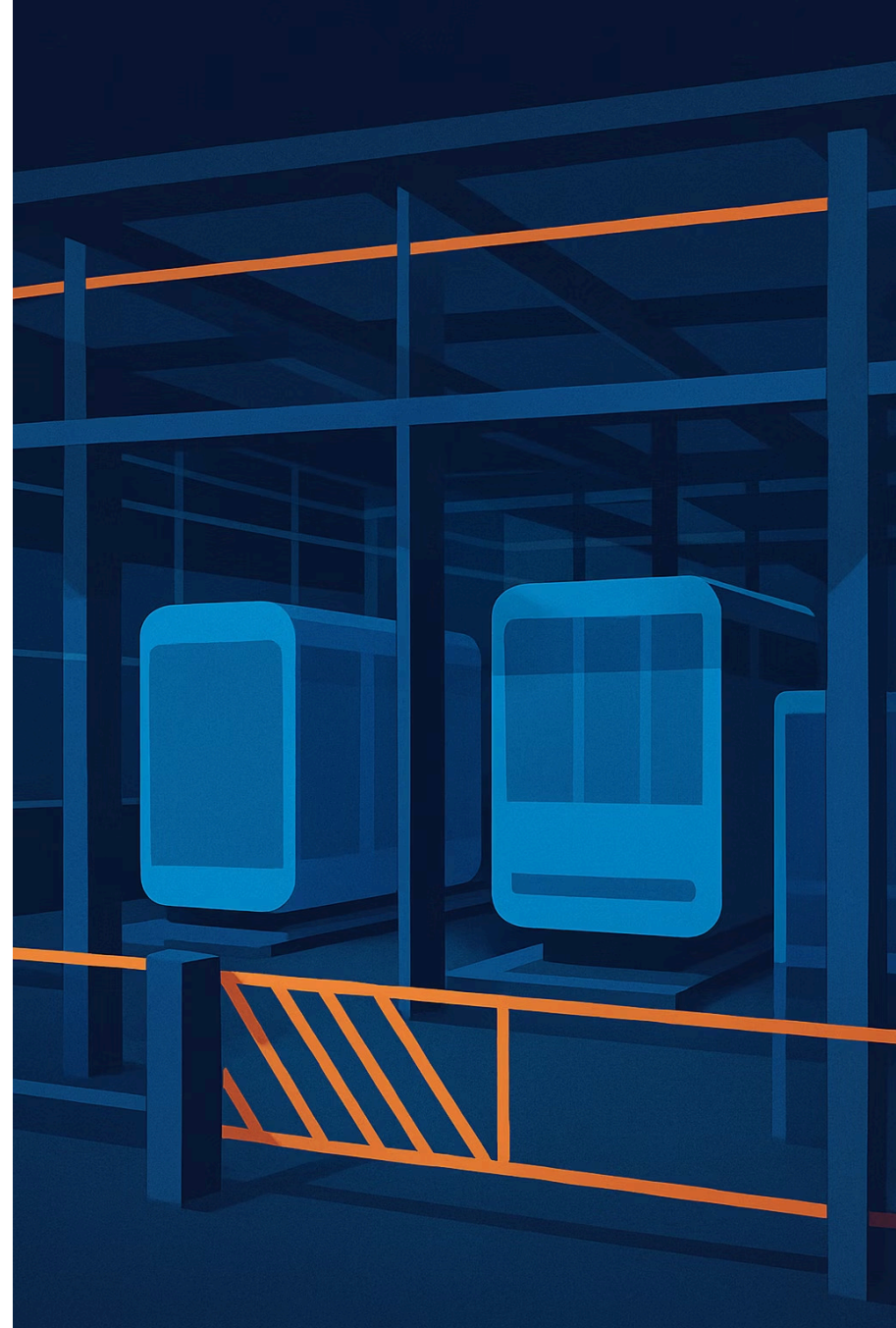
Use both layers. NetworkPolicy alone is not enough for AI workloads.

⚠ **Real scenario:** An AI service with a prompt injection vulnerability reaches your user database because nothing prevents that connection. No breach required at the perimeter.

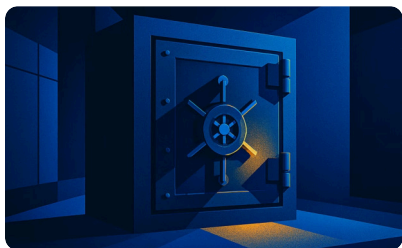
Technical Deep Dive

Isolating AI Systems

Not all services should see the model. Not all traffic should reach the inference endpoint.



AI Isolation: The Core Principles



Model Service Isolation

Deploy model services in dedicated namespaces. Strict ingress: only the gateway or authorized services may call inference endpoints. Deny all else by default.



Prompt & Data Flow Separation

User prompts should never flow directly to model APIs without sanitization. Insert a proxy layer that validates, logs, and rate-limits all inference requests.



Least-Privilege Model Access

Only the services that *explicitly need* model access should have it. Most of your microservices should not even know the model endpoint exists.



Technical Deep Dive

Observability That Actually Matters

Not just logs. Behavioral signals that tell you when something is wrong before the breach happens.

The Right Observability Signals for AI



Latency Anomalies

Sudden latency spikes on inference endpoints indicate unusual prompt complexity or abuse. Baseline and alert.



Token Spikes

Token consumption is a first-class security metric. Exponential growth patterns signal token abuse or runaway inference loops.



Access Patterns

Which services call the model? At what rate? From which identities? Deviation from baseline is a meaningful signal.



Denied Requests

Authorization policy denials are gold. Log them, count them, and alert on spikes – especially from unexpected sources.

DEMO

From Insecure to Secure

A local Kubernetes cluster. A mock AI service. Real Cilium policies. Real results

KIND

CILIUM

HUBBLE

FASTAPI

What to Do Monday Morning

Default deny. Today.

Start with native NetworkPolicy or CiliumNetworkPolicy. Lock down egress and ingress.

Treat AI workloads as sensitive infrastructure

Dedicated namespace, explicit access, labeled boundaries. Not just another deployment.

Make security the easy path

Platform abstractions, GitOps, opinionated defaults. Developers shouldn't have to think about it.

Observe, don't just declare

If you can't see it working, it's not working. Hubble, traffic logs, anomaly detection.

CLOSING THOUGHT

The goal isn't perfect security.

It's building systems where
secure is the default
and fast is still possible

Thank You